DOCUMENTAZIONE TECNICA – INDOVINA CHI?

ENUMERATIVI

ColoriCrapa

```
public enum ColoriCrapa {  43 usages  ripamonti-daniele
    CASTANO, NERO, BIONDO, ROSSO, BIANCO  8 usages
}
```

Rappresenta i possibili colori di capelli dei personaggi del gioco.

ColoriOch

```
public enum ColoriOch {  39 usages  ripamonti-daniele +1
    MARRONE, BLU, VERDE  15 usages
}
```

Rappresenta i possibili colori di capelli dei personaggi del gioco.

ColoriPelle

```
public enum ColoriPelle {  39 usages  ripamonti-daniele
    BIANCO, MULATTO, NERO  22 usages
}
```

Rappresenta i possibili colori di capelli dei personaggi del gioco.

Domanda

```java
public enum Domanda {   43 usages   ripamonti-daniele
    SESSO ( testo: "è maschio?", Categoria.NESSUNA),   1 usage
    CAPELLI_CASTANI( testo: "ha i capelli castani?", Categoria.CAPELLI),   1 usage
    CAPELLI_NERI ( testo: "ha i capelli neri?", Categoria.CAPELLI),   1 usage
    CAPELLI_BIONDI ( testo: "ha i capelli biondi?", Categoria.CAPELLI),   1 usage
    CAPELLI_ROSSI ( testo: "ha i capelli rossi?", Categoria.CAPELLI),   1 usage
    CAPELLI_BIANCHI ( testo: "ha i capelli bianchi?", Categoria.CAPELLI),   1 usage
    PELLE_BIANCA ( testo: "ha la pelle bianca?", Categoria.PELLE),   1 usage
    PELLE_NERA ( testo: "ha la pelle nera?", Categoria.PELLE),   1 usage
    PELLE_MULATTA( testo: "ha la pelle mulatta?", Categoria.PELLE),   1 usage
    OCCHI_MARRONI( testo: "ha gli occhi marroni?", Categoria.OCCHI),   1 usage
    OCCHI_BLU ( testo: "ha gli occhi blu?", Categoria.OCCHI),   1 usage
    OCCHI_VERDI ( testo: "ha gli occhi verdi?", Categoria.OCCHI),   1 usage
    OCCHIALI ( testo: "ha gli occhiali?", Categoria.NESSUNA),   1 usage
    CAPELLI_LUNGHI( testo: "ha i capelli lunghi?", Categoria.NESSUNA),   1 usage
    BARBA ( testo: "ha la barba o i baffi?", Categoria.NESSUNA),   1 usage
    CAPPELLO ( testo: "ha il cappello?", Categoria.NESSUNA),   1 usage
    PELATO ( testo: "è pelato?", Categoria.NESSUNA);   1 usage

    public enum Categoria {   27 usages   ripamonti-daniele
        CAPELLI, OCCHI, PELLE, NESSUNA   5 usages
    }

    private final String testo;   3 usages
    private final Categoria categoria;   2 usages

    Domanda(String testo, Categoria categoria) {   34 usages   ripamonti-daniele
        this.testo = testo;
        this.categoria = categoria;
    }

    public String getTesto() { return testo; }

    public Categoria getCategoria() { return categoria; }

    public boolean corrisponde(Persona p) {   7 usages   ripamonti-daniele
        return switch (this) {
            case SESSO -> p.isSesso();
            case CAPELLI_CASTANI -> p.getColoreCapelli() == ColoriCrapa.CASTANO;
            case CAPELLI_NERI -> p.getColoreCapelli() == ColoriCrapa.NERO;
            case CAPELLI_BIONDI -> p.getColoreCapelli() == ColoriCrapa.BIONDO;
            case CAPELLI_ROSSI -> p.getColoreCapelli() == ColoriCrapa.ROSSO;
            case CAPELLI_BIANCHI -> p.getColoreCapelli() == ColoriCrapa.BIANCO;
            case PELLE_BIANCA -> p.getColorePelle() == ColoriPelle.BIANCO;
            case PELLE_NERA -> p.getColorePelle() == ColoriPelle.NERO;
            case PELLE_MULATTA -> p.getColorePelle() == ColoriPelle.MULATTO;
            case OCCHI_MARRONI -> p.getColoreOcchi() == ColoriOch.MARRONE;
            case OCCHI_BLU -> p.getColoreOcchi() == ColoriOch.BLU;
            case OCCHI_VERDI -> p.getColoreOcchi() == ColoriOch.VERDE;
            case OCCHIALI  -> p.isOcchiali();
            case CAPELLI_LUNGHI -> p.isCapelliLunghi();
            case BARBA -> p.isBarba();
            case CAPPELLO -> p.isCappello();
            case PELATO -> p.isPelato();
        };
    }

    public static Domanda fromTesto(String testo) {   4 usages   ripamonti-daniele
        for (Domanda d : values()) if (d.testo.equals(testo)) return d;
        throw new IllegalArgumentException("Domanda non riconosciuta: " + testo);
    }
}
```

L'enum Domanda rappresenta tutte le domande del gioco, ognuna legata a una caratteristica fisica di una persona (come capelli, occhi o pelle).

Per ogni domanda può verificare se una Persona possiede quella caratteristica e fornire il testo o la categoria associata.

Public getTesto(): restituisce String, il testo della domanda.

Public getCategoria(): restituisce Categoria, la categoria associata alla domanda.

Public corrisponde(Persona p): restituisce boolean, verifica se la persona soddisfa la caratteristica della domanda.

Public static fromTesto(String testo): restituisce Domanda, converte una stringa nella corrispondente domanda; lancia eccezione se non riconosciuta.

CLASSI

PERSONA

Rappresenta un personaggio del gioco con tutte le sue caratteristiche fisiche. La classe è progettata per essere di fatto immutabile dopo la costruzione.

Attributi

- **Private nome (String):** Nome univoco del personaggio.
- **Private coloreCapelli (ColoriCrapa):** Colore dei capelli del personaggio.
- **Private coloreOcchi (ColoriOch):** Colore degli occhi del personaggio.
- **Private colorePelle (ColoriPelle)** Colore della pelle del personaggio.
- **Private occhiali (boolean)** true se il personaggio porta gli occhiali.
- **Private sesso (boolean)** true se il personaggio è maschio.
- **Private capelliLunghi (boolean)** true se il personaggio ha i capelli lunghi.
- **Private barba (boolean)** true se il personaggio ha barba o baffi.
- **Private cappello (boolean)** true se il personaggio indossa un cappello.
- **Private pelato (boolean)** true se il personaggio è pelato.
- **Private nomi (static List)** Lista statica condivisa di tutti i nomi già registrati. Garantisce l'unicità tra le istanze.

Metodi

Persona

```
public Persona(String nome, ColoriCrapa cc, ColoriOch co, ColoriPelle cp, boolean occhiali, boolean sesso, boolean capelliLunghi, boolean barba, boolean cappello, boolean pelato) {  28 usages
    setNome(nome);
    setColoreCapelli(cc);
    setColoreOcchi(co);
    setColorePelle(cp);
    setOcchiali(occhiali);
    setSesso(sesso);
    setCapelliLunghi(capelliLunghi);
    setBarba(barba);
    setCappello(cappello);
    setPelato(pelato);
}
```

Parametri di input

- **nome**: stringa che rappresenta il nome della persona.
- **cc (ColoriCrapa)**: colore dei capelli.
- **co (ColoriOch)**: colore degli occhi.
- **cp (ColoriPelle)**: colore della pelle.
- **occhiali**: valore booleano che indica se la persona porta gli occhiali.
- **sesso**: valore booleano che rappresenta il sesso della persona.
- **capelliLunghi**: indica se la persona ha i capelli lunghi.
- **barba**: indica se la persona ha la barba.
- **cappello**: indica se la persona indossa un cappello.
- **pelato**: indica se la persona è calva.

Parametri di output

- Nessuno

Spiegazione del corpo della funzione

Il costruttore serve a creare un oggetto di tipo **Persona** e a inizializzarne tutte le caratteristiche.

Ogni parametro ricevuto viene passato ai rispettivi metodi set, che assegnano i valori agli attributi interni della classe:

- il nome viene impostato con setNome
- il colore dei capelli, occhi e pelle vengono assegnati tramite i rispettivi setter
- tutte le caratteristiche fisiche (occhiali, sesso, capelli lunghi, barba, cappello, pelato) vengono memorizzate tramite i relativi metodi set

Public Persona

```
public Persona(Persona p) {  1 usage  DANIELE RIPAMONTI +1
    this.nome = p.nome;
    this.coloreCapelli = p.coloreCapelli;
    this.coloreOcchi = p.coloreOcchi;
    this.colorePelle = p.colorePelle;
    this.occhiali = p.occhiali;
    this.sesso = p.sesso;
    this.capelliLunghi = p.capelliLunghi;
    this.barba = p.barba;
    this.cappello = p.cappello;
    this.pelato = p.pelato;
}
```

Parametri di input

- **p (Persona):** oggetto di tipo Persona già esistente, dal quale vengono copiati tutti gli attributi.

Parametri di output

- Nessuno (costruttore di copia, crea una nuova istanza dell'oggetto).

Spiegazione del corpo della funzione

Questo costruttore crea una nuova persona duplicando i valori di un'altra persona già esistente.
Ogni attributo dell'oggetto p viene copiato direttamente nell'oggetto corrente (this).

Non vengono utilizzati metodi setter, ma viene fatta una copia diretta dei campi**.**

Metodi getter

- **Public getNome() :** Restituisce String il nome del personaggio.
- **Public getColoreCapelli() :** Restituisce ColoriCrapa il colore dei capelli.
- **Public getColoreOcchi() :** Restituisce ColoriOch il colore degli occhi.
- **Public getColorePelle() :** Restituisce ColoriPelle il colore della pelle.
- **Public isSesso() :** Restituisce boolean true se maschio.
- **Public isOcchiali() :** Restituisce boolean true se porta gli occhiali.
- **Public isCapelliLunghi() :** Restituisce boolean true se ha i capelli lunghi.
- **Public isBarba() :** Restituisce boolean true se ha barba o baffi.
- **Public isCappello() :** Restituisce boolean true se indossa un cappello.
- **Public isPelato() :** Restituisce boolean true se è pelato.
- **Public getNomi()** : Restituisce List una copia della lista dei nomi registrati

Private readResolve

```
@Serial  no usages  Silvestriignazio
private Object readResolve() {
    if (!nomi.contains(this.nome)) nomi.add(this.nome);
    return this;
}
```

Parametri di input

Nessun parametro esplicito.
Il metodo opera sull'oggetto corrente (this) durante la deserializzazione.

Parametri di output

this (Persona): l'oggetto corrente eventualmente modificato e restituito dopo la deserializzazione.

Spiegazione del corpo della funzione

Questo metodo viene usato durante il processo di deserializzazione di un oggetto (cioè quando un oggetto viene ricostruito da un flusso di dati salvati). Il metodo controlla se il nome della persona (this.nome) è già presente in una lista chiamata nomi.

- Se il nome non è presente, viene aggiunto alla lista nomi.
- Se invece è già presente, non viene fatto nulla.

NODO (ABSTRACT)

Classe base astratta per i nodi dell'albero binario decisionale. Ogni nodo possiede un ID intero auto-incrementale, un figlio sinistro ("sì") e uno destro ("no").

gestisce correttamente il contatore statico durante la deserializzazione tramite readresolve().

Attributi

- **Private contatore (static, int):** Contatore statico condiviso usato per generare ID progressivi univoci.
- **Private id (int):** Identificatore univoco del nodo, assegnato al momento della creazione.
- **Private si (Nodo):** Riferimento al figlio corrispondente alla risposta "sì".
- **Private no (Nodo):** Riferimento al figlio corrispondente alla risposta "no"

Metodi

Metodi getter e setter

- **Public getId():** Restituisce int l'ID univoco del nodo.
- **Public getSi():** Restituisce Nodo il figlio "sì".
- **Public getNo()** : Restituisce Nodo il figlio "no".
- **Private setSi(Nodo n)** : Restituisce void, imposta il figlio "sì". Lancia IllegalStateException se già occupato.
- **Private setNo(Nodo n)** :Restituisce void, imposta il figlio "no". Lancia IllegalStateException se già occupato.
- **Public isFoglia()** : Restituisce boolean true se il nodo non ha figli.
- **Private readResolve()** : Object Aggiorna il contatore statico in base all'ID corrente durante la deserializzazione.

NODO DOMANDA (ESTENSIONE DI NODO)

Nodo interno dell'albero che contiene il testo di una domanda. Può avere due figli (ramo "sì" e ramo "no"). Il testo viene normalizzato in minuscolo e vengono tolti gli spazi; la lunghezza deve essere compresa tra 1 e 40 caratteri.

Attributi

- **Private String domanda:** testo della domanda
- **Private Static final long serialVersionUID:** id per la deserializzazione

Metodi

- **Public NodoDomanda(String domanda):** Restituisce Stringa domanda, crea il nodo validando e normalizzando il testo della domanda.
- **Public getDomanda():** Restituisce (String) il testo della domanda.
- **toString():** Restituisce (String) la rappresentazione di testo.

NODO PERSONA (ESTENSIONE DI NODO)

Nodo foglia dell'albero che contiene un riferimento a una Persona. Sovrascrive setSi() e setNo() lanciando UnsupportedOperationException per impedire l'aggiunta di figli.

Attributi

- **private final persona (Persona):** persona associata al nodo foglia

Metodi

- **Public NodoPersona(Persona p):** Crea il nodo con la persona indicata. Lancia IllegalArgumentException se null.
- **Public getPersona():** restituisce la persona
- **Public setSi(Nodo n):** restituisce errore se il nodo persona non può avere figli
- **Public setNo(Nodo n):** restituisce errore se il nodo persona non può avere figli

Albero

La classe è un albero decisionale binario, questo è composto dai nodi domanda e i nodi foglia (persona), l'albero è navigabile tramite il bot che risponde alle domande Si/No.

Attributi

Implementa Serializable per consentire il salvataggio su file .ser della struttura dati.

SerialVersionUID (long): identificatore di versione per la serializzazione

Root (NodoDomanda): Nodo radice dell'albero

Indice (Map<Integer, Nodo>): Dizionario che associa a ogni ID al corrispondente nodo

Metodi

Albero

```
public Albero(String domandaRoot) {  1 usage  ripamonti-daniele +1
    root = new NodoDomanda(domandaRoot);
    indice.put(root.getId(), root);
}
```

Parametri di input

- **DomandaRoot (stringa)**: che rappresenta la domanda iniziale (radice) dell'albero.

Parametri di output

- Nessuno (costruttore, inizializza la struttura dati).

Spiegazione del corpo della funzione

Il costruttore crea un nuovo oggetto di tipo Albero inizializzando la sua struttura principale.

- Viene creato un nodo radice (root) di tipo NodoDomanda, utilizzando la stringa domandaRoot.
- Questo nodo rappresenta il punto di partenza dell'albero.
- Successivamente, il nodo radice viene inserito in una struttura dati chiamata indice, che associa l'ID del nodo (root.getId()) al nodo stesso.

Public InserisciDomanda

```
public int inserisciDomanda(int idPadre, String domanda, boolean si) {  1 usage  ripamonti-daniele
    NodoDomanda nd = new NodoDomanda(domanda);
    collegaNodo(idPadre, nd, si);
    return nd.getId();
}
```

Parametri di input

- **idPadre**: intero che rappresenta l'ID del nodo padre a cui collegare la nuova domanda.
- **domanda**: stringa che rappresenta il contenuto della nuova domanda da inserire nell'albero.
- **si**: valore booleano che indica su quale ramo collegare il nuovo nodo (es. ramo "sì" o "no").

Parametri di output

- **int**: ID del nuovo nodo domanda creato e inserito nell'albero.

Spiegazione del corpo della funzione

La funzione crea un nuovo nodo di tipo NodoDomanda utilizzando la stringa domanda.

Successivamente:

- il nuovo nodo viene collegato al nodo padre identificato da idPadre
- il collegamento avviene tramite il metodo collegaNodo, che usa il parametro booleano si per decidere il ramo (ad esempio ramo sì o ramo no)

Infine, viene restituito l'ID del nodo appena creato tramite nd.getId().

Public inserisciPersona

```
public int inserisciPersona(int idPadre, Persona persona, boolean si) {  2 usages  ripamonti-daniele
    NodoPersona np = new NodoPersona(persona);
    collegaNodo(idPadre, np, si);
    return np.getId();
}
```

Parametri di input

- **idPadre**: intero che rappresenta l'ID del nodo padre a cui collegare la nuova persona.
- **persona**: oggetto di tipo Persona che contiene i dati della persona da inserire.
- **si**: valore booleano che indica su quale ramo collegare il nuovo nodo (es. ramo "sì" o "no").

Parametri di output

- **int**: ID del nuovo nodo persona creato e inserito nell'albero.

Spiegazione del corpo della funzione

La funzione crea un nuovo nodo di tipo NodoPersona utilizzando l'oggetto persona.

Successivamente:

- il nuovo nodo viene collegato al nodo padre identificato da idPadre
- il collegamento avviene tramite il metodo collegaNodo, che usa il parametro booleano si per decidere il ramo (ad esempio ramo sì o ramo no)

Infine, viene restituito l'ID del nodo appena creato tramite np.getId().

Private collegaNodo

```
private void collegaNodo(int idPadre, Nodo nuovo, boolean si) {  2 usages  ripamonti-daniele
    Nodo padre = indice.get(idPadre);
    if (padre == null) throw new IllegalArgumentException("Nessun nodo con id " + idPadre);
    if (!(padre instanceof NodoDomanda)) throw new IllegalStateException("Solo i nodi domanda possono avere figli");
    if (si) padre.setSi(nuovo);
    else padre.setNo(nuovo);
    indice.put(nuovo.getId(), nuovo);
}
```

Parametri di input

- **idPadre**: intero che rappresenta l'ID del nodo padre a cui collegare il nuovo nodo.
- **nuovo (Nodo)**: nodo da inserire nell'albero (può essere una domanda o una persona).
- **si**: valore booleano che indica su quale ramo collegare il nodo (ramo "sì" oppure "no").

Parametri di output

- Nessuno (metodo void, modifica direttamente la struttura dell'albero)

Spiegazione del corpo della funzione

La funzione collega un nuovo nodo a un nodo padre già presente nell'albero.

- Prima recupera il nodo padre dall'indice usando idPadre.
- Se il padre non esiste, viene generato un errore (IllegalArgumentException).
- Se il nodo padre non è un NodoDomanda, viene generato un errore (IllegalStateException), perché solo le domande possono avere figli.

Poi:

- se si è vero, il nuovo nodo viene collegato come figlio del ramo "sì"
- altrimenti viene collegato come ramo "no"
- il nuovo nodo viene aggiunto alla mappa indice, usando il suo ID come chiave.

Getters pubblici

- **getRootId():** int Restituisce l'ID del nodo radice.
- **getRoot():** Restituisce NodoDomanda Restituisce il riferimento diretto alla radice.
- **getDomande():** Restituisce una mappa <ID, String> con il testo di tutte le domande presenti nell'albero.
- **getPersone():** Restituisce una mappa <ID, Persona> di tutte le persone foglia presenti nell'albero.

BOT

Costruisce un albero decisionale basato su domande sì/no per distinguere un insieme di personaggi. Seleziona automaticamente le domande (in modo casuale o ottimizzato) per dividere progressivamente le possibili persone. Durante l'esecuzione, percorre l'albero in base alle risposte fino a individuare il personaggio scelto.

Attributi

Implementa Serializable per permettere il salvataggio dello stato del bot su file .ser.

serialVersionUID (long): identificatore di versione per la serializzazione.

opzioniSI (Map<Domanda, List<Persona>>): associa a ogni domanda la lista di persone che rispondono "sì".

opzioniNO (Map<Domanda, List<Persona>>): associa a ogni domanda la lista di persone che rispondono "no".

tutte (List<Persona>): contiene tutte le persone disponibili nel gioco.

personaggioDaScoprire (Persona): rappresenta il personaggio scelto casualmente che il bot deve far indovinare.

nodoCorrente (Nodo): indica la posizione attuale nel percorso dell'albero decisionale.

albero (Albero): struttura dati che rappresenta l'intero albero di domande e risposte.

Metodi

Metodi getter e setter

- **Public getRoot(): restituisce Nodo, la radice dell'albero decisionale.**
- **Public getPersonaggioDaScoprire(): restituisce Persona, il personaggio scelto dal bot.**
- **Public getDomandaCorrente(): restituisce String, il testo della domanda del nodo corrente (se presente).**
- **Private setPersone(List<Persona> persone): restituisce void, inizializza la lista delle persone copiandole ed evitando valori null (lancia eccezione se la lista non è valida).**
- **Private setPersonaggioDaScoprire(): restituisce void, seleziona casualmente il personaggio da indovinare.**

public Bot

```
public Bot(List<Persona> persone, boolean difficile) {  2 usages  ripamonti-daniele
    setPersone(persone);
    setPersonaggioDaScoprire();
    inizializza(difficile);
}
```

Parametri di input

- **persone (List<Persona>):** lista di oggetti Persona tra cui il bot sceglierà il personaggio da indovinare.
- **difficile (boolean):** valore che indica il livello di difficoltà del bot.

Parametri di output

- Nessuno (costruttore, inizializza l'oggetto Bot).

Spiegazione del corpo della funzione

Il costruttore inizializza un oggetto Bot eseguendo tre operazioni principali:

Chiama setPersone(persone) per assegnare al bot la lista delle persone disponibili.

Chiama setPersonaggioDaScoprire() per scegliere (probabilmente in modo casuale) una persona dalla lista che il giocatore dovrà indovinare.
Chiama inizializza(difficile) per configurare il comportamento del bot in base al livello di difficoltà scelto.

public Bot (costruttore di copia)

```
public Bot(List<Persona> persone) {  1 usage  ripamonti-daniele
    this(persone, difficile: false);
}
```

Parametri di input

- **persone (List<Persona>):** lista di oggetti Persona tra cui il bot sceglierà il personaggio.

Parametri di output

- Nessuno (costruttore, inizializza l'oggetto Bot).

Spiegazione del corpo della funzione

Questo costruttore richiama un altro costruttore della stessa classe passando:

- la lista delle persone ricevuta
- il valore false come livello di difficoltà

In pratica, delega tutta l'inizializzazione al costruttore principale che gestisce anche la difficoltà.

public void inizializza

```
private void inizializza(boolean difficile) {  1 usage  ripamonti-daniele +1
    for (Domanda d : Domanda.values()) {
        List<Persona> si = new ArrayList<>();
        List<Persona> no = new ArrayList<>();
        for (Persona p : tutte) {
            if (d.corrisponde(p)) si.add(p);
            else no.add(p);
        }
        opzioniSI.put(d, si);
        opzioniNO.put(d, no);
    }
    creaAlbero(difficile);
}
```

Parametri di input

- **difficile (boolean):** indica il livello di difficoltà da utilizzare nella configurazione del bot.

Parametri di output

- Nessuno (metodo void, modifica lo stato interno dell'oggetto).

Spiegazione del corpo della funzione

Il metodo prepara tutte le strutture necessarie per il funzionamento del bot. Scorre tutte le possibili domande tramite Domanda.values(). Per ogni domanda crea due liste:

- si: contiene le persone che soddisfano la domanda
- no: contiene le persone che non la soddisfano

Per ogni persona nella lista tutte, verifica la condizione usando d.corrisponde(p):

- se la persona corrisponde alla domanda viene inserita in si
- altrimenti viene inserita in no

Le due liste vengono poi salvate in due strutture:

- opzioniSI associa la domanda alle persone che rispondono "sì"
- opzioniNO associa la domanda alle persone che rispondono "no"

Infine viene chiamato creaAlbero(difficile) per costruire la struttura ad albero in base alla difficoltà scelta.

Private Domanda domandaMigliore

```java
private Domanda domandaMigliore(List<Domanda> disponibili, List<Persona> personeCorrente) {  2 usages  ripamonti-daniele
    Domanda migliore = null;
    int entropia = Integer.MAX_VALUE;
    for (Domanda d : disponibili) {
        List<Persona> si = interseca(personeCorrente, opzioniSI.get(d));
        List<Persona> no = interseca(personeCorrente, opzioniNO.get(d));
        int entr = Math.abs(si.size() - no.size());
        if (entr == 0 && !si.isEmpty() && !no.isEmpty()) return d;
        if (entr < entropia) {
            migliore = d;
            entropia = entr;
        }
    }
    return migliore;
}
```

Parametri di input

- **disponibili (List<Domanda>):** lista delle domande che possono ancora essere utilizzate.
- **personeCorrente (List<Persona>):** lista delle persone ancora possibili in base alle risposte date finora.

Parametri di output

- **Domanda:** la domanda ritenuta migliore da porre.

Spiegazione del corpo della funzione

Il metodo cerca la domanda più efficace tra quelle disponibili e inizializza una variabile migliore a null e un valore entropia molto alto.

Per ogni domanda:

- Calcola due liste:
  - si: persone correnti che risponderebbero “sì”
  - no: persone correnti che risponderebbero “no”
    Questo avviene facendo l’intersezione tra personeCorrente e le liste già calcolate (opzioniSI e opzioniNO).
- Calcola una misura chiamata entr, che è la differenza assoluta tra la dimensione delle due liste.
  Se la differenza è 0 e entrambe le liste non sono vuote, restituisce subito quella domanda (è perfettamente bilanciata).
  Altrimenti, se entr è minore del valore minimo trovato finora:
  - aggiorna migliore
  - aggiorna entropia

Alla fine restituisce la domanda migliore trovata.

Private void creaAlbero

```java
private void creaAlbero(boolean difficile) {  1 usage  ripamonti-daniele +1
    List<Domanda> domandeList = new ArrayList<>(Arrays.asList(Domanda.values()));
    Domanda domandaRoot;
    if (difficile) domandaRoot = domandaMigliore(domandeList, tutte);

    else {
        Collections.shuffle(domandeList);
        domandaRoot = domandeList.getFirst();
    }
    domandeList.remove(domandaRoot);

    albero = new Albero(domandaRoot.getTesto());
    nodoCorrente = albero.getRoot();

    Set<Domanda> usate = new HashSet<>();
    usate.add(domandaRoot);

    Set<Domanda.Categoria> categorieConfermateSI = new HashSet<>();
    if (domandaRoot.getCategoria() != Domanda.Categoria.NESSUNA) categorieConfermateSI.add(domandaRoot.getCategoria());

    costruisciSottoAlbero(albero.getRootId(),  rispostaPadre: true, interseca(tutte, opzioniSI.get(domandaRoot)), new HashSet<>(usate), new HashSet<>(categorieConfermateSI), difficile);

    costruisciSottoAlbero(albero.getRootId(),  rispostaPadre: false, interseca(tutte, opzioniNO.get(domandaRoot)), new HashSet<>(usate), new HashSet<>(), difficile);
}
```

Parametri di input

- **difficile (boolean):** indica se il bot deve costruire l'albero in modalità difficile o normale.

Parametri di output

- Nessuno (metodo void, costruisce e aggiorna la struttura interna dell'albero).

Spiegazione del corpo della funzione

La funzione costruisce l'albero decisionale del bot. Prima crea una lista con tutte le domande disponibili e poi seleziona la domanda iniziale (radice):

- se difficile è vero, viene scelta la domanda migliore usando un criterio di ottimizzazione
- altrimenti le domande vengono mescolate e viene scelta la prima

Successivamente la domanda radice viene rimossa dalla lista delle disponibili.

Viene creato un nuovo albero usando il testo della domanda scelta come radice e il nodo corrente viene impostato su quello iniziale.

Vengono inizializzati:

- un insieme di domande già utilizzate, contenente la radice
- un insieme di categorie confermate "sì", eventualmente aggiornato in base alla domanda radice

Infine la funzione costruisce ricorsivamente i due rami dell'albero:

- ramo "sì": usando le persone che soddisfano la domanda
- ramo "no": usando le persone che non la soddisfano

Per ogni ramo viene chiamata la funzione costruisciSottoAlbero, passando lo stato aggiornato (domande usate, categorie confermate e lista filtrata delle persone).

Private void costruisciSottoAlbero

```java
private void costruisciSottoAlbero(int idPadre, boolean rispostaPadre, List<Persona> personeCorrente, Set<Domanda> usate, Set<Domanda.Categoria> categorieConfermate, boolean difficile) {  4 usages
    if (personeCorrente.isEmpty()) return;

    if (personeCorrente.size() == 1) {
        albero.inserisciPersona(idPadre, personeCorrente.getFirst(), rispostaPadre);
        return;
    }

    List<Domanda> disponibili = new ArrayList<>();
    for (Domanda d : Domanda.values()) {
        if (usate.contains(d)) continue;
        if (d.getCategoria() != Domanda.Categoria.NESSUNA && categorieConfermate.contains(d.getCategoria())) continue;
        List<Persona> si = interseca(personeCorrente, opzioniSI.get(d));
        List<Persona> no = interseca(personeCorrente, opzioniNO.get(d));
        if (!si.isEmpty() && !no.isEmpty()) disponibili.add(d);
    }

    if (disponibili.isEmpty()) {
        for (Persona p : personeCorrente) albero.inserisciPersona(idPadre, p, rispostaPadre);
        return;
    }

    Domanda scelta = difficile ? domandaMigliore(disponibili, personeCorrente) : disponibili.get(new Random().nextInt(disponibili.size()));
    int idNuovo = albero.inserisciDomanda(idPadre, scelta.getTesto(), rispostaPadre);

    Set<Domanda> nuoveUsate = new HashSet<>(usate);
    nuoveUsate.add(scelta);

    Set<Domanda.Categoria> confermateSI = new HashSet<>(categorieConfermate);
    if (scelta.getCategoria() != Domanda.Categoria.NESSUNA) confermateSI.add(scelta.getCategoria());

    costruisciSottoAlbero(idNuovo, rispostaPadre: true, interseca(personeCorrente, opzioniSI.get(scelta)), new HashSet<>(nuoveUsate), confermateSI, difficile);
    costruisciSottoAlbero(idNuovo, rispostaPadre: false, interseca(personeCorrente, opzioniNO.get(scelta)), new HashSet<>(nuoveUsate), new HashSet<>(categorieConfermate), difficile);
}
```

Parametri di input

- **idPadre (int):** identificativo del nodo padre a cui collegare il nuovo nodo.
- **rispostaPadre (boolean):** indica se il collegamento al padre avviene tramite ramo “sì” (true) o “no” (false).
- **personeCorrente (List<Persona>):** insieme delle persone ancora possibili in quel ramo dell’albero.
- **usate (Set<Domanda>):** insieme delle domande già utilizzate nel percorso corrente.
- **categorieConfermate (Set<Domanda.Categoria>)**: categorie già confermate che non devono essere riutilizzate.
- **difficile (boolean):** indica se la selezione delle domande deve essere ottimizzata o casuale.

Parametri di output

- Nessuno (metodo void, modifica la struttura dell’albero).

Spiegazione del corpo della funzione

La funzione costruisce ricorsivamente i rami dell’albero decisionale.

- Se personeCorrente è vuota, la funzione termina subito perché non ci sono elementi da rappresentare.
- Se c’è una sola persona nella lista:
  - viene inserito direttamente un nodo foglia nell’albero tramite inserisciPersona
  - il ramo viene chiuso
- Se ci sono più persone:
  - viene creata la lista disponibili con le domande ancora utilizzabili
  - per ogni domanda:
    - → vengono escluse quelle già usate
    - → vengono escluse quelle appartenenti a categorie già confermate
    - → viene controllato che la domanda divida effettivamente il gruppo in due parti non vuote (sì/no)
- Se non ci sono domande valide:
  - tutte le persone rimanenti vengono inserite come foglie nel nodo corrente

    - la funzione termina
- Se invece esistono domande valide:
    - viene scelta una domanda:
        - → se difficile è true, viene scelta la migliore con criterio ottimizzato
        - → altrimenti viene scelta casualmente
    - la domanda viene inserita nell'albero come nuovo nodo figlio del padre
- Vengono aggiornati:
    - le domande già usate (aggiungendo la nuova scelta)
    - le categorie confermate (se la domanda appartiene a una categoria valida)
- Infine la funzione richiama sé stessa due volte:
    - ramo "sì": con le persone che soddisfano la domanda scelta
    - ramo "no": con le persone che non la soddisfano

Private List<Persona> intrinseca

```
private List<Persona> interseca(List<Persona> a, List<Persona> b) {  8 usages  EnissLaZone +1
    List<Persona> risultato = new ArrayList<>();
    for (Persona pA : a) {
        for (Persona pB : b) {
            if (pA.getNome().equals(pB.getNome())) {
                risultato.add(pA);
                break;
            }
        }
    }
    return risultato;
}
```

Parametri di input

- **a (List<Persona>):** prima lista di persone.
- **b (List<Persona>):** seconda lista di persone.

Parametri di output

- **List<Persona>:** lista contenente le persone presenti in entrambe le liste (intersezione).

Spiegazione del corpo della funzione

La funzione costruisce una nuova lista contenente solo le persone che sono presenti sia nella lista a che nella lista b.

- Viene creata una lista vuota chiamata risultato.
- Per ogni persona nella lista a:
  - si confronta con ogni persona nella lista b.
  - il confronto avviene tramite il nome (getNome()), quindi due persone sono considerate uguali se hanno lo stesso nome.
- Se viene trovata una corrispondenza:
  - la persona viene aggiunta alla lista risultato
  - si interrompe il ciclo interno per evitare duplicazioni

Alla fine viene restituita la lista con tutte le persone comuni alle due liste.

SERIALIZZATORE

La classe Serializzatore gestisce il salvataggio e il caricamento di bot, persone e stato della partita su file .ser. Permette di interrompere e riprendere il gioco mantenendo tutte le informazioni necessarie tramite serializzazione.

Attributi

PERCORSO_SALVATAGGIO (String): indica il percorso del file .ser utilizzato per salvare e caricare lo stato della partita.

Metodi

Public static deSerializzaPersone

```java
public static List<Persona> deSerializzaPersone(String percorso) {
    try {
        FileInputStream file = new FileInputStream(percorso);
        ObjectInputStream input = new ObjectInputStream(file);
        Object o = input.readObject();
        input.close();
        file.close();
        if (o instanceof List<?>) {
            if (((List<?>) o).isEmpty()) return null;
            if (((List<?>) o).stream().allMatch( capture of ? e -> e instanceof Persona)) return (List<Persona>) o;
        }
        return null;
    }
    catch (IOException e) {
        String p = "files/persone.ser";
        if (!percorso.equals(p)) return deSerializzaPersone(p);
        throw new RuntimeException("Errore nella deSerializzazione del file " + percorso);
    }
    catch (ClassNotFoundException e) {
        String p = "files/persone.ser";
        if (!percorso.equals(p)) return deSerializzaPersone(p);
        throw new RuntimeException("Errore: non è stato deSerializzato un Object");
    }
}
```

Parametri di input

- **percorso (String):** percorso del file da cui leggere un oggetto serializzato contenente una lista di persone.

Parametri di output

- **List<Persona>:** lista di persone deserializzate dal file, oppure null se il contenuto non è valido o la lista è vuota.

Spiegazione del corpo della funzione

La funzione legge un file serializzato e prova a ricostruire una lista di oggetti Persona.

- Apre un FileInputStream sul percorso indicato.
- Lo passa a un ObjectInputStream per leggere un oggetto serializzato.
- Legge l'oggetto dal file con readObject().
- Chiude gli stream.

Poi controlla il contenuto letto:

- verifica che l'oggetto sia una lista (List<?>)
- se la lista è vuota, restituisce null
- verifica che tutti gli elementi della lista siano di tipo Persona
- se i controlli sono validi, effettua il cast e restituisce la lista

Se avviene un errore di I/O (IOException):

- prova automaticamente a leggere un file di fallback: "files/persone.ser"
- se anche quel percorso è già quello usato, lancia un’eccezione runtime

Se avviene un errore di classe non trovata (ClassNotFoundException):

- applica lo stesso meccanismo di fallback
- oppure lancia un’eccezione runtime se il fallback non è possibile

Public static deSerializzaBot

```java
public static Bot deSerializzaBot(String percorso) {  3 usages  ripamonti-daniele +1
    try {
        FileInputStream file = new FileInputStream(percorso);
        ObjectInputStream input = new ObjectInputStream(file);
        Object o = input.readObject();
        input.close();
        file.close();
        if (o instanceof Bot) return (Bot) o;
        return null;
    }
    catch (IOException e) {
        throw new RuntimeException("Errore nella deSerializzazione del file " + percorso);
    }
    catch (ClassNotFoundException e) {
        throw new RuntimeException("Errore: non è stato deSerializzato un Object");
    }
}
```

Parametri di input

- **percorso (String):** percorso del file da cui leggere l'oggetto serializzato di tipo Bot.

Parametri di output

- **Bot:** oggetto Bot deserializzato dal file, oppure null se il contenuto non è di tipo Bot.

Spiegazione del corpo della funzione

La funzione legge un file contenente un oggetto serializzato e prova a ricostruire un'istanza della classe Bot.

- Apre un FileInputStream sul percorso indicato.
- Lo collega a un ObjectInputStream per leggere oggetti serializzati.
- Legge l'oggetto dal file tramite readObject().
- Chiude gli stream per liberare le risorse.

Dopo la lettura:

- controlla se l'oggetto deserializzato è di tipo Bot
  - se sì, lo restituisce effettuando il cast
  - se no, restituisce null

Se si verifica un errore di input/output (IOException):

- viene lanciata un'eccezione RuntimeException con un messaggio che indica il file coinvolto

Se si verifica un errore di classe non trovata (ClassNotFoundException):

- viene lanciata un'eccezione RuntimeException indicando che il tipo dell'oggetto non è stato riconosciuto

Public static serializzaPartita

```
public static void serializzaPartita(Object[] stato) {  2 usages  EnissLaZone
    try {
        FileOutputStream file = new FileOutputStream(PERCORSO_SALVATAGGIO);
        ObjectOutputStream output = new ObjectOutputStream(file);
        output.writeObject(stato);
        output.close();
        file.close();
    }
    catch (IOException e) {
        throw new RuntimeException("Errore nel salvataggio della partita: " + e.getMessage());
    }
}
```

Parametri di input

- **stato (Object[]):** array di oggetti che rappresenta lo stato completo della partita da salvare.

Parametri di output

- Nessuno (metodo void, salva i dati su file).

Spiegazione del corpo della funzione

La funzione salva lo stato della partita su file tramite serializzazione.

- Apre un FileOutputStream utilizzando il percorso definito da PERCORSO_SALVATAGGIO.
- Collega il flusso a un ObjectOutputStream, che permette di scrivere oggetti su file.
- Scrive l'intero array stato nel file con writeObject.
- Chiude entrambi gli stream per completare correttamente il salvataggio e liberare risorse.

Se durante il processo si verifica un errore di input/output (IOException):

- viene lanciata un'eccezione RuntimeException
- il messaggio include la descrizione dell'errore per facilitare il debug

Public static deserializzaPartita

```
public static Object[] deSerializzaPartita() {  1 usage  EnissLaZone
    try {
        FileInputStream file = new FileInputStream(PERCORSO_SALVATAGGIO);
        ObjectInputStream input = new ObjectInputStream(file);
        Object o = input.readObject();
        input.close();
        file.close();
        if (o instanceof Object[]) return (Object[]) o;
        return null;
    }
    catch (IOException e) {
        throw new RuntimeException("Nessuna partita salvata trovata.");
    }
    catch (ClassNotFoundException e) {
        throw new RuntimeException("File di salvataggio non valido.");
    }
}
```

Parametri di input

- Nessuno (il metodo legge direttamente dal file di salvataggio definito da PERCORSO_SALVATAGGIO).

Parametri di output

- Object[]: array che rappresenta lo stato della partita deserializzato dal file, oppure null se il contenuto non è valido.

Spiegazione del corpo della funzione

La funzione recupera lo stato di una partita salvata in precedenza leggendo un file serializzato.

- Apre un FileInputStream sul percorso di salvataggio.
- Collega il flusso a un ObjectInputStream per leggere oggetti serializzati.
- Legge l'oggetto salvato tramite readObject().
- Chiude gli stream per liberare le risorse.

Dopo la lettura:

- controlla se l'oggetto è di tipo Object[]
  - se sì, lo restituisce dopo il cast
  - se no, restituisce null

Se si verifica un errore di input/output (IOException):

- viene lanciata un'eccezione RuntimeException con il messaggio "Nessuna partita salvata trovata."

Se si verifica un errore di classe non trovata (ClassNotFoundException):

- viene lanciata un'eccezione RuntimeException che indica che il file di salvataggio non è valido

SCHERMATA GIOCO

La classe SchermataGioco gestisce tutta l'interfaccia grafica del gioco "IndovinaChi", includendo menu, partita e gestione delle carte.
Permette di giocare in locale o contro un bot, gestendo turni, domande e stato della partita.

Attributi

- **private final List<Persona> persone:** contiene tutte le persone disponibili nel gioco da indovinare.
- **private final Domanda[] domandePossibili:** array delle domande utilizzabili durante la partita.
- **private Bot bot:** rappresenta il motore di gioco contro il computer (null in modalità solo giocatori).
- **private boolean difficile:** indica se il bot è in modalità difficile (uso di strategia ottimizzata).

Metodi

inizializzaMenu(): crea la schermata principale con le modalità di gioco, caricamento partita ed uscita.

inizializzaInPersona(): avvia la modalità a due giocatori e permette di scegliere le rispettive persone segrete.

avviaPartitaInPersona(...): gestisce tutta la logica della partita tra due giocatori (turni, domande, eliminazioni e vittoria).

inizializzaBot(): avvia la modalità contro il computer scegliendo le persone segrete e configurando il bot.

avviaPartitaBot(...): gestisce la partita giocatore vs bot, inclusi turni, domande e logica dell'albero decisionale.

caricaPartita(): carica uno stato salvato e ripristina la partita precedente.

salvaPartitaInPersona(...): salva lo stato della partita in modalità due giocatori.

salvaPartitaBot(...): salva lo stato della partita contro il bot.

ripristinaInPersona(Object[] stato): ricostruisce una partita tra due giocatori da un salvataggio.

ripristinaBot(Object[] stato): ricostruisce una partita contro il bot da un salvataggio.

aggiornaCombo(...): aggiorna le domande disponibili in base alle risposte già date.

aggiornaUI(...): aggiorna lo stato grafico dell'interfaccia (turno, bottoni e colori).

eseguiTurnoBot(...): gestisce il turno del bot, facendo avanzare l'albero e decidere la mossa.

mostraSceltaPersona(int numeroGiocatore): permette a un giocatore di scegliere la propria persona segreta.

creaGriglia(...): costruisce la griglia grafica delle carte dei personaggi.

abbattiCarta(...): elimina visivamente una carta quando un personaggio viene escluso.

finePartita(...): mostra la schermata finale e permette di rigiocare o tornare al menu.

categoriaConfermata(...): verifica se una categoria è già stata confermata da risposte precedenti.

personeRaggiungibili(...): restituisce tutte le persone ancora possibili in un ramo dell'albero.

raccogliPersone(...): metodo di supporto per raccogliere ricorsivamente le persone da un nodo.

Public void avviaPartitaInPersona

```java
private void avviaPartitaInPersona(Persona personaSegretaG1, Persona personaSegretaG2, int turnoIniziale, List<String> abbattuteG1, List<String> abbattuteG2, List<String> domandeFatteG1, List<String> domandeFatteG2) {
    final int[] turno = {turnoIniziale};

    List<Boolean> risposteG1 = new ArrayList<>();
    List<Boolean> risposteG2 = new ArrayList<>();

    Map<String, JPanel> cartaPerNomeG1 = new HashMap<>();
    Map<String, JPanel> cartaPerNomeG2 = new HashMap<>();

    JPanel pannelloPrincipale = new JPanel(new BorderLayout());
    pannelloPrincipale.setBackground(new Color( r: 158,  g: 26,  b: 14));
    setContentPane(pannelloPrincipale);

    JPanel colonnaG1 = creaColonna( titolo: "Giocatore 1", creaGriglia(cartaPerNomeG1),  bordo: true);
    JPanel colonnaG2 = creaColonna( titolo: "Giocatore 2", creaGriglia(cartaPerNomeG2),  bordo: false);

    JPanel pannelloCentrale = new JPanel(new GridLayout( rows: 1,  cols: 2,  hgap: 0,  vgap: 0));
    pannelloCentrale.setOpaque(false);
    pannelloCentrale.add(colonnaG1);
    pannelloCentrale.add(colonnaG2);

    for (String nome : abbattuteG1) abbattiCarta(nome, cartaPerNomeG1);
    for (String nome : abbattuteG2) abbattiCarta(nome, cartaPerNomeG2);

    JPanel pannelloLaterale = new JPanel(new BorderLayout());
    pannelloLaterale.setBackground(new Color( r: 180,  g: 180,  b: 180));
    pannelloLaterale.setPreferredSize(new Dimension( width: 220,  height: 0));

    JPanel headerLaterale = new JPanel(new BorderLayout());
    headerLaterale.setBackground(new Color( r: 30,  g: 100,  b: 200));
    headerLaterale.setBorder(BorderFactory.createEmptyBorder( top: 10,  left: 10,  bottom: 10,  right: 10));
    JLabel labelTurno = new JLabel( text: "Turno: Giocatore " + turnoIniziale, SwingConstants.CENTER);
    labelTurno.setForeground(Color.WHITE);
    labelTurno.setFont(labelTurno.getFont().deriveFont(Font.BOLD,  size: 14f));
    headerLaterale.add(labelTurno, BorderLayout.CENTER);

    JPanel pannelloControlli = new JPanel();
    pannelloControlli.setLayout(new BoxLayout(pannelloControlli, BoxLayout.Y_AXIS));
    pannelloControlli.setBackground(new Color( r: 180,  g: 180,  b: 180));
    pannelloControlli.setBorder(BorderFactory.createEmptyBorder( top: 12,  left: 15,  bottom: 20,  right: 15));

    JComboBox<String> comboDomande = new JComboBox<>();
    comboDomande.setMaximumSize(new Dimension( width: 200,  height: 28));
    comboDomande.setAlignmentX(Component.CENTER_ALIGNMENT);

    JButton btnFaiDomanda = creaBottoneLaterale( testo: "Fai domanda", new Color( r: 30,  g: 100,  b: 200));
    JButton btnIndovina = creaBottoneLaterale( testo: "Tenta di indovinare!", new Color( r: 46,  g: 160,  b: 67));
    JButton btnSalva = creaBottoneLaterale( testo: "Salva partita", new Color( r: 160,  g: 100,  b: 0));
```

```java
    pannelloControlli.add(btnIndovina);
    pannelloControlli.add(Box.createVerticalStrut( height: 10));
    pannelloControlli.add(comboDomande);
    pannelloControlli.add(Box.createVerticalStrut( height: 8));
    pannelloControlli.add(btnFaiDomanda);
    pannelloControlli.add(Box.createVerticalStrut( height: 10));
    pannelloControlli.add(btnSalva);

    pannelloLaterale.add(headerLaterale, BorderLayout.NORTH);
    pannelloLaterale.add(pannelloControlli, BorderLayout.CENTER);

    pannelloPrincipale.add(pannelloCentrale, BorderLayout.CENTER);
    pannelloPrincipale.add(pannelloLaterale, BorderLayout.EAST);

    aggiornaCombo(turno[0], domandeFatteG1, risposteG1, domandeFatteG2, risposteG2, comboDomande);

    btnSalva.addActionListener( ActionEvent _ -> salvaPartitaInPersona(personaSegretaG1, personaSegretaG2, turno[0], abbattuteG1, abbattuteG2, domandeFatteG1, domandeFatteG2));

    btnIndovina.addActionListener( ActionEvent _ -> {
        int turnoCorrente = turno[0];
        String input = JOptionPane.showInputDialog( parentComponent: this,  message: "Giocatore " + turnoCorrente + ", inserisci il nome della persona:",  title: "Tenta di indovinare", JOptionPane.QUESTION_MESSAGE);
        if (input == null) return;

        String nomeInput = input.trim().toLowerCase();
        boolean esiste = false;
        for (Persona p : persone) {
            if (p.getNome().equals(nomeInput)) {
                esiste = true;
                break;
            }
        }
        if (!esiste) {
            JOptionPane.showMessageDialog( parentComponent: this,  message: "\"" + nomeInput + "\" non è un personaggio valido.\nControlla l'ortografia e riprova.",  title: "Nome non valido", JOptionPane.WARNING_MESSAGE);
            return;
        }

        Persona segreta = (turnoCorrente == 1) ? personaSegretaG2 : personaSegretaG1;
        if (segreta.getNome().equals(nomeInput)) {
            JOptionPane.showMessageDialog( parentComponent: this,  message: "Giocatore " + turnoCorrente + " ha vinto!\nLa persona era: " + segreta.getNome(),  title: "Hai vinto!", JOptionPane.INFORMATION_MESSAGE);
            finePartita(pannelloLaterale,  bot: false);
        }
        else {
            if (turnoCorrente == 1) turno[0] = 2;
            else turno[0] = 1;
            labelTurno.setText("Turno: Giocatore " + turno[0]);
            headerLaterale.setBackground(new Color( r: 30,  g: 100,  b: 200));
            aggiornaCombo(turno[0], domandeFatteG1, risposteG1, domandeFatteG2, risposteG2, comboDomande);
```

```java
    Persona segretaAvversario;
    if (avversario == 1) segretaAvversario = personaSegretaG1;
    else segretaAvversario = personaSegretaG2;

    boolean rispostaCorretta = domanda.corrisponde(segretaAvversario);
    boolean rispostaUtente;

    do {
        int sc = JOptionPane.showOptionDialog( parentComponent: this,  message: "Giocatore " + avversario + ", rispondi alla domanda:\n\"" + testoDomanda + "\"",  title: "Rispondi", JOptionPane.DEFAULT_OPTION, JOptionPane.QUESTION_MESSAGE,  icon: nu
        if (sc == JOptionPane.CLOSED_OPTION) return;
        rispostaUtente = (sc == 0);
        if (rispostaUtente != rispostaCorretta) {
            JOptionPane.showMessageDialog( parentComponent: this,  message: "Risposta sbagliata! Devi rispondere correttamente.",  title: "Errore", JOptionPane.ERROR_MESSAGE);
        }
    } while (rispostaUtente != rispostaCorretta);

    domandeFatteCorrente.add(testoDomanda);
    risposteCorrenti.add(rispostaCorretta);

    Map<String, JPanel> cartaDaAggiornare;
    List<String> abbattuteDaAggiornare;

    if (turnoCorrente == 1) {
        cartaDaAggiornare = cartaPerNomeG1;
        abbattuteDaAggiornare = abbattuteG1;
    }
    else {
        cartaDaAggiornare = cartaPerNomeG2;
        abbattuteDaAggiornare = abbattuteG2;
    }
    for (Persona p : persone) {
        if (domanda.corrisponde(p) != rispostaCorretta) {
            abbattiCarta(p.getNome(), cartaDaAggiornare);
            if (!abbattuteDaAggiornare.contains(p.getNome())) abbattuteDaAggiornare.add(p.getNome());
        }
    }

    turno[0] = avversario;
    labelTurno.setText("Turno: Giocatore " + turno[0]);
    aggiornaCombo(turno[0], domandeFatteG1, risposteG1, domandeFatteG2, risposteG2, comboDomande);
});

revalidate();
repaint();
```

Parametri di input

- **personaSegretaG1 (Persona):** persona segreta assegnata al giocatore 1.
- **personaSegretaG2 (Persona):** persona segreta assegnata al giocatore 2.
- **turnoIniziale (int):** indica quale giocatore inizia la partita.
- **abbattuteG1 (List<String>):** elenco dei nomi delle carte eliminate dal giocatore 1.
- **abbattuteG2 (List<String>):** elenco dei nomi delle carte eliminate dal giocatore 2.
- **domandeFatteG1 (List<String>):** domande già fatte dal giocatore 1.
- **domandeFatteG2 (List<String>):** domande già fatte dal giocatore 2.

Parametri di output

- Nessuno (metodo void, inizializza e gestisce l'interfaccia grafica della partita).

Spiegazione del corpo della funzione

La funzione avvia e gestisce l'interfaccia grafica di una partita tra due giocatori.

- Inizializza il turno utilizzando un array modificabile.
- Crea strutture per memorizzare:
  - risposte dei giocatori
  - associazione tra nomi e pannelli grafici delle carte per entrambi i giocatori
- Imposta il pannello principale della finestra con layout e colori.
- Crea due colonne di gioco (Giocatore 1 e Giocatore 2) con le rispettive griglie di carte.
- Applica lo stato iniziale eliminando le carte già abbattute.
- Crea un pannello laterale contenente:
  - indicatore del turno corrente
  - combo box con le domande disponibili
  - pulsanti per fare domanda, indovinare e salvare la partita
- Collega il pannello centrale e quello laterale alla finestra.
- Aggiorna la combo delle domande disponibili in base al turno.

Gestione eventi:

- Pulsante “Salva partita”
  - Salva lo stato corrente della partita richiamando salvaPartitaInPersona.
- Pulsante “Tenta di indovinare”: chiede al giocatore di inserire il nome della persona. Verifica che il nome sia valido tra le persone disponibili.
  Se il nome è corretto rispetto alla persona segreta dell’avversario:
  - mostra messaggio di vittoria
  - termina la partita

  Se è sbagliato:

  - cambia turno
  - aggiorna interfaccia e domande disponibili

Pulsante “Fai domanda”

- Recupera la domanda selezionata dalla combo box.
- Identifica giocatore corrente e avversario.
- Determina la persona segreta dell’avversario.
- Verifica la risposta corretta della domanda rispetto alla persona segreta.
- Richiede all’avversario di rispondere correttamente tramite finestra di dialogo.
- Se la risposta è errata:
  - ripete la richiesta finché non viene data correttamente
- Aggiorna:
- lista domande fatte
- lista risposte
- stato delle carte (eliminando quelle incompatibili con la risposta)
- Cambia turno e aggiorna interfaccia.

Public void avviaPartitaBot

```java
private void avviaPartitaBot(Persona personaSegreta, Persona personaSegretaBot, int turnoIniziale, List<String> abbattuteGiocatore, List<String> abbattuteBot, List<String> domandeFatteGiocatore, Nodo nodoIniziale, Bot bot, boolean difficile) {
    getContentPane().removeAll();
    JPanel root = new JPanel(new BorderLayout());
    root.setBackground(new Color( r: 158,  g: 26,  b: 14));
    setContentPane(root);

    Nodo[] sceltaCorrente = {nodoIniziale};
    List<Boolean> risposteGiocatore = new ArrayList<>();
    int[] turno = {turnoIniziale};

    Map<String, JPanel> cartaPerNomeGiocatore = new HashMap<>();
    Map<String, JPanel> cartaPerNomeBot2      = new HashMap<>();

    JPanel colonnaGiocatore = creaColonna( titolo: "Giocatore", creaGriglia(cartaPerNomeGiocatore),  bordo: true);
    JPanel colonnaBot = creaColonna( titolo: "Bot", creaGriglia(cartaPerNomeBot2), bordo: false);

    JPanel pannelloCentrale = new JPanel(new GridLayout( rows: 1,  cols: 2,  hgap: 0,  vgap: 0));
    pannelloCentrale.setOpaque(false);
    pannelloCentrale.add(colonnaGiocatore);
    pannelloCentrale.add(colonnaBot);

    for (String nome : abbattuteGiocatore) abbattiCarta(nome, cartaPerNomeGiocatore);
    for (String nome : abbattuteBot) abbattiCarta(nome, cartaPerNomeBot2);

    // Pannello laterale con header fisso in cima
    JPanel pannelloLaterale = new JPanel(new BorderLayout());
    pannelloLaterale.setBackground(new Color( r: 180,  g: 180,  b: 180));
    pannelloLaterale.setPreferredSize(new Dimension( width: 220,  height: 0));

    JPanel headerLaterale = new JPanel(new BorderLayout());
    headerLaterale.setBackground(new Color( r: 30,  g: 100,  b: 200));
    headerLaterale.setBorder(BorderFactory.createEmptyBorder( top: 10,  left: 10,  bottom: 10,  right: 10));
    JLabel labelTurno = new JLabel( text: "Turno: Giocatore", SwingConstants.CENTER);
    labelTurno.setForeground(Color.WHITE);
    labelTurno.setFont(labelTurno.getFont().deriveFont(Font.BOLD,  size: 14f));
    headerLaterale.add(labelTurno, BorderLayout.CENTER);

    JPanel pannelloControlli = new JPanel();
    pannelloControlli.setLayout(new BoxLayout(pannelloControlli, BoxLayout.Y_AXIS));
    pannelloControlli.setBackground(new Color( r: 180,  g: 180,  b: 180));
    pannelloControlli.setBorder(BorderFactory.createEmptyBorder( top: 12,  left: 15,  bottom: 20,  right: 15));

    JComboBox<String> comboDomande = new JComboBox<>();
    comboDomande.setMaximumSize(new Dimension( width: 200,  height: 28));
    comboDomande.setAlignmentX(Component.CENTER_ALIGNMENT);

    JButton btnFaiDomanda = creaBottoneLaterale( testo: "Fai domanda", new Color( r: 30,  g: 100,  b: 200));
    JButton btnIndovina = creaBottoneLaterale( testo: "Tenta di indovinare!", new Color( r: 44,  g: 160,  b: 47));
    JButton btnSalva = creaBottoneLaterale( testo: "Salva partita", new Color( r: 140,  g: 100,  b: 0));
```

```java
JLabel labelSegreta = new JLabel( text: "La tua persona segreta:");
labelSegreta.setAlignmentX(Component.CENTER_ALIGNMENT);
labelSegreta.setBorder(BorderFactory.createEmptyBorder( top: 10,  left: 0,  bottom: 8,  right: 0));

JPanel cartaSegreta = new JPanel(new BorderLayout( hgap: 0,  vgap: 4));
cartaSegreta.setBackground(new Color( r: 210,  g: 210,  b: 210));
cartaSegreta.setBorder(BorderFactory.createLineBorder(Color.YELLOW,  thickness: 2));
cartaSegreta.setMaximumSize(new Dimension( width: 110,  height: 150));
cartaSegreta.setAlignmentX(Component.CENTER_ALIGNMENT);
JLabel imgSegreta = new JLabel(getImmagine(personaSegreta.getNome(),  larghezza: 90,  altezza: 110));
imgSegreta.setHorizontalAlignment(SwingConstants.CENTER);
cartaSegreta.add(imgSegreta, BorderLayout.CENTER);

pannelloControlli.add(comboDomande);
pannelloControlli.add(Box.createVerticalStrut( height: 8));
pannelloControlli.add(btnFaiDomanda);
pannelloControlli.add(Box.createVerticalStrut( height: 8));
pannelloControlli.add(btnIndovina);
pannelloControlli.add(Box.createVerticalStrut( height: 8));
pannelloControlli.add(btnSalva);
pannelloControlli.add(Box.createVerticalGlue());
pannelloControlli.add(labelSegreta);
pannelloControlli.add(cartaSegreta);

pannelloLaterale.add(headerLaterale, BorderLayout.NORTH);
pannelloLaterale.add(pannelloControlli, BorderLayout.CENTER);

root.add(pannelloCentrale, BorderLayout.CENTER);
root.add(pannelloLaterale, BorderLayout.EAST);

comboDomande.removeAllItems();
for (Domanda d : domandePossibili) {
    if (!domandeFatteGiocatore.contains(d.getTesto()) && !categoriaConfermata(d, domandeFatteGiocatore, risposteGiocatore)) comboDomande.addItem(d.getTesto());
}

aggiornaUI(turno[0], comboDomande, btnFaiDomanda, btnIndovina, btnSalva, labelTurno, headerLaterale);

btnSalva.addActionListener( ActionEvent _ -> salvaPartitaBot(personaSegreta, personaSegretaBot, turno[0], abbattuteGiocatore, abbattuteBot, domandeFatteGiocatore, sceltaCorrente[0], difficile));

btnFaiDomanda.addActionListener( ActionEvent _ -> {
    String testoDomanda = (String) comboDomande.getSelectedItem();
    if (testoDomanda == null) return;

    Domanda domanda  = Domanda.fromTesto(testoDomanda);
    boolean risposta = domanda.corrisponde(personaSegretaBot);
    JOptionPane.showMessageDialog( parentComponent: this,  message: "Bot risponde: " + (risposta ? "Sì" : "No"),  title: "Risposta del bot", JOptionPane.INFORMATION_MESSAGE);

    domandeFatteGiocatore.add(testoDomanda);
    risposteGiocatore.add(risposta);

    for (Persona p : persone) {
        if (domanda.corrisponde(p) != risposta) {
            abbattiCarta(p.getNome(), cartaPerNomeGiocatore);
            if (!abbattuteGiocatore.contains(p.getNome())) abbattuteGiocatore.add(p.getNome());
        }
```

```java
    }

    comboDomande.removeAllItems();
    for (Domanda d : domandePossibili) {
        if (!domandeFatteGiocatore.contains(d.getTesto()) && !categoriaConfermata(d, domandeFatteGiocatore, risposteGiocatore)) comboDomande.addItem(d.getTesto());
    }
    turno[0] = 2;
    aggiornaUI(turno[0], comboDomande, btnFaiDomanda, btnIndovina, btnSalva, labelTurno, headerLaterale);
    eseguiTurnoBot(personaSegreta, cartaPerNomeBot2, abbattuteBot, turno, sceltaCorrente, pannelloLaterale);
    aggiornaUI(turno[0], comboDomande, btnFaiDomanda, btnIndovina, btnSalva, labelTurno, headerLaterale);
});

btnIndovina.addActionListener( ActionEvent _ -> {
    String input = JOptionPane.showInputDialog( parentComponent: this, message: "Inserisci il nome della persona segreta del bot:", title: "Tenta di indovinare", JOptionPane.QUESTION_MESSAGE);
    if (input == null) return;

    String nomeInput = input.trim().toLowerCase();
    boolean esiste = persone.stream().anyMatch( Persona p -> p.getNome().equals(nomeInput));
    if (!esiste) {
        JOptionPane.showMessageDialog( parentComponent: this, message: "\"" + nomeInput + "\" non è un personaggio valido.", title: "Nome non valido", JOptionPane.WARNING_MESSAGE);
        return;
    }
    if (personaSegretaBot.getNome().equals(nomeInput)) {
        JOptionPane.showMessageDialog( parentComponent: this, message: "Hai vinto! La persona del bot era: " + personaSegretaBot.getNome(), title: "Hai vinto!", JOptionPane.INFORMATION_MESSAGE);
        finePartita(pannelloLaterale, bot: true);
    }
    else {
        JOptionPane.showMessageDialog( parentComponent: this, message: "Sbagliato! Tocca al bot.", title: "Risposta errata", JOptionPane.WARNING_MESSAGE);
        turno[0] = 2;
        aggiornaUI(turno[0], comboDomande, btnFaiDomanda, btnIndovina, btnSalva, labelTurno, headerLaterale);
        eseguiTurnoBot(personaSegreta, cartaPerNomeBot2, abbattuteBot, turno, sceltaCorrente, pannelloLaterale);
        aggiornaUI(turno[0], comboDomande, btnFaiDomanda, btnIndovina, btnSalva, labelTurno, headerLaterale);
    }
});

if (turnoIniziale == 2) {
    aggiornaUI(turno[0], comboDomande, btnFaiDomanda, btnIndovina, btnSalva, labelTurno, headerLaterale);
    eseguiTurnoBot(personaSegreta, cartaPerNomeBot2, abbattuteBot, turno, sceltaCorrente, pannelloLaterale);
}
aggiornaUI(turno[0], comboDomande, btnFaiDomanda, btnIndovina, btnSalva, labelTurno, headerLaterale);

revalidate();
repaint();
```

Parametri di input

- **personaSegreta (Persona):** persona segreta del giocatore umano.
- **personaSegretaBot (Persona):** persona segreta assegnata al bot.
- **turnoIniziale (int):** indica quale tra giocatore e bot inizia la partita.
- **abbattuteGiocatore (List<String>):** nomi delle carte eliminate dal giocatore.
- **abbattuteBot (List<String>):** nomi delle carte eliminate dal bot.
- **domandeFatteGiocatore (List<String>):** elenco delle domande già fatte dal giocatore.
- **nodoIniziale (Nodo):** nodo di partenza dell'albero decisionale del bot.
- **bot (Bot):** istanza del bot utilizzata per la logica di gioco.
- **difficile (boolean):** indica se il bot usa una strategia più ottimizzata o semplice.
-

Parametri di output

- Nessuno (metodo void, gestisce l'intera partita e l'interfaccia grafica).

Spiegazione del corpo della funzione

La funzione avvia una partita tra un giocatore umano e un bot, gestendo interfaccia grafica e logica di gioco.

- Pulisce la finestra e crea il layout principale.
- Inizializza:
  - nodo corrente dell'albero decisionale
  - lista risposte del giocatore
  - gestione del turno
- Crea le strutture grafiche:
  - colonna giocatore

    - colonna bot
    - pannello centrale con entrambe le colonne
- Aggiorna lo stato iniziale eliminando le carte già abbattute.
- Crea il pannello laterale con:
    - indicatore turno
    - combo box delle domande disponibili
    - pulsanti di gioco (domanda, indovina, salva)
    - visualizzazione della carta segreta del giocatore
- Popola la combo delle domande filtrando:
    - domande già fatte
    - domande escluse da categorie già confermate
- Aggiorna l'interfaccia in base al turno iniziale.

Gestione eventi

- Pulsante "Salva partita": salva lo stato corrente della partita bot tramite salvaPartitaBot.
- Pulsante "Fai domanda":
    - Recupera la domanda selezionata.
    - Il bot risponde in base alla sua persona segreta.
    - Registra la domanda e la risposta.
    - Aggiorna la griglia del giocatore eliminando carte incompatibili.
    - Aggiorna la lista delle domande disponibili.
    - Passa il turno al bot.
    - Il bot esegue automaticamente il suo turno tramite eseguiTurnoBot.
    - Aggiorna l'interfaccia dopo ogni azione.
- Pulsante "Tenta di indovinare": il giocatore inserisce il nome della persona segreta del bot. Viene verificata la validità del nome.

  Se è corretto:
    - il giocatore vince la partita
    - la partita termina

  Se è sbagliato:

    - passa il turno al bot
    - il bot esegue la sua mossa automaticamente